Java Maps & Sets Ultimate Cheat Sheet

Think of Java Collections like different kinds of lockers:

- **Hash** → fastest chaos ⚡
- **Linked** → remembers insertion order 📝
- **Tree** → always sorted 🌲

1. MAPS in Java

A **Map** stores:

`key -> value`

Example:

`101 -> "Gurparshad"`

Keys are unique.

MAP FAMILY OVERVIEW

Type	Ordering	Internal DS	Time Complexity	Null Keys	Best Use
`HashMap`	No order	Hash Table	O(1) avg	1 null key	Fast access
`LinkedHashMap`	Insertion order	Hash Table + Doubly Linked List	O(1) avg	1 null key	Ordered fast map
`TreeMap`	Sorted order	Red Black Tree	O(log n)	❌	Sorted data

2. HASHMAP ⚡

```
HashMap<Integer, String> map = new HashMap<>();
```

Internal Implementation Idea

```
Array of buckets
+
Hashing
+
Linked List / Tree inside bucket
```

How it works

```
key.hashCode()
      ↓
bucket index
      ↓
store value
```

If collisions happen:

```
same bucket
↓
Linked List
↓
Too many collisions?
↓
Converted to Red Black Tree
```

(Java 8+)

Characteristics

- ✅ Fastest map
- ✅ No ordering
- ✅ Allows one null key
- ✅ Not synchronized

Time Complexity

Operation	Complexity
put()	O(1) avg
get()	O(1) avg
remove()	O(1) avg
Worst case	O(n)

Example

```
HashMap<Integer, String> map = new HashMap<>();

map.put(1, "A");
map.put(2, "B");

System.out.println(map.get(1));
```

3. LINKEDHASHMAP 📜

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
```

Internal Implementation Idea

```
HashMap
+
Doubly Linked List
```

It stores insertion order using linked list pointers.

Characteristics

✅ Maintains insertion order
✅ Almost as fast as HashMap
✅ Allows one null key

Time Complexity

Operation	Complexity
put()	O(1)
get()	O(1)
remove()	O(1)

Example

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>();

map.put(3, "C");
map.put(1, "A");
map.put(2, "B");

System.out.println(map);
```

Output:

```
{3=C, 1=A, 2=B}
```

Insertion order preserved 🎯

4. TREEMAP 🎄

```
TreeMap<Integer, String> map = new TreeMap<>();
```

Internal Implementation Idea

```
Red Black Tree
(Self Balancing BST)
```

Every key remains sorted automatically.

Characteristics

- ✅ Sorted keys
- ✅ No null keys
- ✅ Slower than HashMap
- ✅ Great for ranges/sorted data

Time Complexity

Operation	Complexity
put()	O(log n)
get()	O(log n)
remove()	O(log n)

Example

```
TreeMap<Integer, String> map = new TreeMap<>();

map.put(3, "C");
map.put(1, "A");
map.put(2, "B");

System.out.println(map);
```

Output:

```
{1=A, 2=B, 3=C}
```

Automatically sorted 🎄

MAP DIFFERENCE SUMMARY ⚔️

Feature	HashMap	LinkedHashMap	TreeMap
Ordering	❌	Insertion order	Sorted
Speed	Fastest	Fast	Slower
DS Used	Hash Table	Hash + DLL	Red Black Tree
Null Key	✅	✅	❌
Complexity	O(1)	O(1)	O(log n)
Best For	Fast lookup	Ordered lookup	Sorted data

5. SETS in Java

A **Set** stores:

```
Unique values only
```

No duplicates allowed.

SET FAMILY OVERVIEW

Type	Ordering	Internal DS	Complexity	Null Allowed
`HashSet`	No order	HashMap	O(1)	1 null
`LinkedHashSet`	Insertion order	LinkedHashMap	O(1)	1 null
`TreeSet`	Sorted	TreeMap	O(log n)	❌

6. HASHSET ⚡

```
HashSet<Integer> set = new HashSet<>();
```

Internal Implementation Idea

`Built using HashMap`

Actually stores:

`value -> dummy object`

Example internally:

```
5 -> PRESENT
10 -> PRESENT
```

Characteristics

✅ Fastest set
✅ No ordering
✅ Unique values only

Time Complexity

Operation	Complexity
add()	O(1)
contains()	O(1)
remove()	O(1)

Example

```
HashSet<Integer> set = new HashSet<>();

set.add(5);
set.add(1);
set.add(5);

System.out.println(set);
```

Duplicates removed automatically ⚡

7. LINKEDHASHSET 📜

```
LinkedHashSet<Integer> set = new LinkedHashSet<>();
```

Internal Implementation Idea

```
LinkedHashMap internally
```

Characteristics

✅ Insertion order maintained
✅ Unique elements
✅ Slightly slower than HashSet

Example

```
LinkedHashSet<Integer> set = new LinkedHashSet<>();

set.add(3);
set.add(1);
set.add(2);

System.out.println(set);
```

Output:

```
[3, 1, 2]
```

8. TREESET 🎄

```
TreeSet<Integer> set = new TreeSet<>();
```

Internal Implementation Idea

```
TreeMap internally
+
Red Black Tree
```

Characteristics

✅ Sorted elements
✅ No duplicates
✅ No null values

Time Complexity

Operation	Complexity
add()	O(log n)
contains()	O(log n)
remove()	O(log n)

Example

```
TreeSet<Integer> set = new TreeSet<>();

set.add(3);
set.add(1);
set.add(2);

System.out.println(set);
```

Output:

```
[1, 2, 3]
```

Sorted automatically 🌲

SET DIFFERENCE SUMMARY ⚔️

Feature	HashSet	LinkedHashSet	TreeSet
Ordering	❌	Insertion order	Sorted
Speed	Fastest	Fast	Slower
Internal DS	HashMap	LinkedHashMap	TreeMap
Null Allowed	✅	✅	❌
Complexity	O(1)	O(1)	O(log n)

GOLDEN MEMORY TRICK 🧠🔥

HASH = SPEED

```
HashMap / HashSet
↓
Use when only speed matters
```

LINKED = ORDER

```
LinkedHashMap / LinkedHashSet
↓
Use when insertion order matters
```

TREE = SORTING

```
TreeMap / TreeSet
↓
Use when sorted data matters
```

INTERVIEW KILLER LINE 🎯

```
HashMap gives fastest average lookup using hashing,
LinkedHashMap maintains insertion order using DLL,
TreeMap keeps keys sorted using Red Black Tree.
```

MOST IMPORTANT INTERNALS 🌋

Collection	Actually Uses
HashSet	HashMap

LinkedHashSet	LinkedHashMap
TreeSet	TreeMap
TreeMap	Red Black Tree
HashMap	Hash Table
LinkedHashMap	Hash Table + DLL

WHEN TO USE WHAT 🚀

Situation	Best Choice
Fast lookup	HashMap / HashSet
Maintain insertion order	LinkedHashMap / LinkedHashSet
Sorted data	TreeMap / TreeSet
Leaderboards/rankings	TreeMap
Cache systems	LinkedHashMap
Frequency counting	HashMap

SUPER SHORT REVISION ⚡

```
Hash -> Fast
Linked -> Ordered
Tree -> Sorted
```

Java collections in one spell-scroll. 🧙‍♂️